FINAL YEAR B.E. CAPSTONE PROJECT EXAMINATION

AgriChain: Complete Technical Architecture, Codebase & Viva Voce Master Reference Guide

A comprehensive reference manual covering all libraries, backend microservices, machine learning anomaly detection algorithms, smart contract governance, zero-float integer conventions, gas optimizations, and examiner-level questions & answers.

Section 1: Master Technology Stack & Library Matrix

AgriChain is built on an enterprise 5-tier architecture spanning hardware/simulated edge devices, asynchronous Python microservices, unsupervised machine learning, modular EVM Solidity smart contracts, and a responsive React frontend. The following matrix outlines every library and technology utilized across the project.

Layer / Domain	Technology & Version	Core Functionality in AgriChain	Engineering Justification vs Alternatives
Backend REST API	FastAPI (v0.115+)	Asynchronous ASGI web framework managing telemetry ingestion, batch registrations, AI pipeline routing, and role views.	300% faster than Flask/Django; native ASGI concurrency for high-frequency IoT streaming; automatic Swagger/OpenAPI docs and Rust-powered Pydantic serialization.
ASGI Web Server	Uvicorn (v0.30+)	Lightning-fast ASGI server running the FastAPI application on top of the high-performance uvloop event loop.	Production-grade asynchronous non-blocking I/O capable of handling hundreds of concurrent IoT sensor pings without thread pool starvation.
Data Validation	Pydantic (v2.9+)	Strict request/response payload schema parsing, type coercion, and data sanitization.	Rejects corrupt or malformed IoT sensor packets at the API boundary before they consume CPU cycles or enter application memory.
Machine Learning	Scikit-Learn (v1.5+)	Implements the unsupervised Isolation Forest anomaly detection algorithm and evaluation metrics.	Highly optimized C-accelerated tree partitioning; executes sub-15ms inference latency on standard CPU hardware without requiring GPU infrastructure.
Numerical Math	NumPy & Pandas (v1.26+)	Vectorized mathematical transformations, time-series moving averages, and rolling z-scores.	Enables instant multi-sensor feature engineering (first-order temperature gradients, humidity ratios, and z-score anomaly vectors).
Database & ORM	SQLite & SQLAlchemy / SQLModel	Relational persistence of raw incoming telemetry (readings table), active batches, custody events, and audit logs.	Implements the strict 'Persistence-Before-Validation' rule to guarantee complete auditability, with zero configuration overhead and seamless Supabase PostgreSQL migration.
EVM Blockchain Client	Web3.py (v7.0+)	Python library for interacting with Ethereum/Polygon Amoy JSON-RPC	Industry-standard EVM abstraction for transaction signing, ABI

		nodes.	encoding, bitwise struct packing, and smart contract method invocation.
Sensor Cryptography	eth-account & pycryptodome	Hardware ECDSA public/private key signature verification and SHA-256 IsoLog hash chaining.	Verifies IoT edge identity directly via secp256k1 signature recovery, while cryptographically chaining quarantined records off-chain.
Smart Contracts	Solidity (v0.8.28)	Modular smart contracts executing custody state transitions, SLA penalty deductions, and escrow settlements.	Latest stable version with native integer overflow checks and custom errors for maximum gas efficiency.
Development Framework	Hardhat (v2.22+)	Ethereum development environment for contract compilation, local EVM node simulation, and testing.	Fast automated unit testing (22 passing tests), local chain fork inspection, and automated gas profiling.
Contract Primitives	OpenZeppelin Contracts (v5.0+)	Industry-audited implementations of AccessControl (RBAC) and Pausable (Emergency Circuit Breakers).	Mitigates common smart contract exploits (reentrancy, unauthorized writes) using standardized, mathematically proven access control primitives.
Frontend UI	React (v18.3+)	Component-based Single Page Application (SPA) managing role-based dashboards (Farmer, Logistics, Dark Store, Consumer, Admin).	Fast Virtual DOM rendering, reactive component lifecycle hooks, and real-time custom event synchronization across views.
Build Tool	Vite (v6.0+)	Next-generation frontend build tool and development server using native ES modules.	Sub-second Hot Module Replacement (HMR) and 4.38-second production builds compiling 1,817 modules.
Styling System	Tailwind CSS (v3.4+)	Utility-first CSS framework enforcing a locked design token system.	Zero runtime stylesheet overhead, highly consistent cross-dashboard layout, and strict design adherence without third-party UI libraries.
Web3 Client (Frontend)	Ethers.js (v6.17+)	Client-side EVM blockchain library for MetaMask connectivity and contract calls.	Modern, lightweight Web3 library with native BigInt support for integer currency (paise) and clean asynchronous contract abstractions.

Section 2: End-to-End Operational Pipeline (Step-by-Step Data Journey)

To understand how AgriChain works in practice, examine the life cycle of a single environmental reading taken by an IoT edge sensor installed inside a refrigerated transit truck (reefer) carrying organic tomatoes:

- **Step 1: Telemetry Ingestion & Raw Persistence —** The sensor captures temperature, humidity, GPS coordinates, and timestamp, attaches its hardware ECDSA signature, and sends an HTTP POST request to /telemetry on the FastAPI gateway. Following our strict Persistence-Before-Validation rule, the raw payload is saved to the SQLite 'readings' table immediately, ensuring an immutable forensic record even if the packet is later flagged as fraudulent.
- **Step 2: Cryptographic Identity & Anti-Replay Check —** The pipeline passes the reading to integrity.py. The system verifies that the timestamp is strictly monotonic ($t_k > t_{k-1}$) and has not exceeded the 15-minute

heartbeat limit (t_k - t_{k-1} <= 900s). It then uses eth-account to recover the signer address from the ECDSA signature, confirming that the data originated from an authentic, registered hardware device rather than a network impersonator.

- **Step 3: Deterministic Physical Plausibility & Crop Policy Filter —** The reading is evaluated against physical laws in plausibility.py and biological policies in policy.py. It verifies that the thermal rate of change does not exceed |ΔT/Δt| <= 1.5°C/minute (the physical limit of insulated reefers) and that the truck's speed (calculated via the Haversine formula) does not exceed 120 km/h. Concurrently, it checks crop bounds (tomatoes: 2.0°C to 8.0°C).
- **Step 4: Multivariate Feature Extraction & Isolation Forest ML —** The pipeline in features.py computes a multi-dimensional feature vector: temperature gradient, humidity rate of change, cross-sensor ratio (Temp/Humidity), and a rolling statistical z-score across the preceding 5 samples. These features are fed into the Scikit-learn Isolation Forest model, which measures tree isolation path lengths to detect subtle calibration drifts, frozen sensor outputs, or multivariate anomalies.
- **Step 5: Composite Verdict & IsoLog Quarantine Chaining —** The verdict.py engine synthesizes all checks into a final status: VALID, BREACH, or ANOMALOUS. If anomalous, the payload is immediately blocked from touching the blockchain (WRITE_BLOCKED). Instead, it is directed to audit.py, which chains the incident record into an off-chain SHA-256 IsoLog Hash Chain (entry_hash = SHA256(prev_hash + reading_id + reasons + timestamp)), providing complete tamper-evident auditability with zero gas cost.
- **Step 6: Single-Slot Bitwise Struct Packing & On-Chain Oracle Execution —** If the reading is VALID or a legitimate biological BREACH, chain.py bit-packs the metrics into a single 256-bit word and dispatches a signed transaction via Web3.py to ColdChainMonitor.sol. If a breach occurred, the contract automatically computes dynamic penalty deductions and logs them against the logistics escrow payout.
- **Step 7: Multi-Stakeholder UI & Consumer QR Reflection —** The backend broadcasts updates to frontend clients. The Farmer Dashboard displays the batch trust score (92/100); the Logistics Dashboard updates vehicle tracking; the Dark Store receives inbound shipment allocations; and consumers scanning the product QR code can view the entire verified 5-stage timeline and fair price breakdown.

Section 3: Granular File-by-File & Program Breakdown

Every file in the AgriChain codebase has a dedicated responsibility within the modular architecture. The following detailed breakdown describes the exact imports, classes, functions, and logic implemented in each program.

3.1 Backend Core Services (backend/)

backend/main.py — Central API Gateway & System Orchestrator

• Key Imports: fastapi, pydantic, hashlib, numpy, time, datetime, chain, ipfs, db, and trust-layer service modules.
• Core Endpoints & Logic:
- POST /telemetry: Main ingestion endpoint. Accepts TelemetryPayload, executes raw SQLite insertion, orchestrates the AI Trust Layer pipeline, and routes data to either blockchain anchoring or quarantine.
- GET /farmer/ai-trust: Aggregates recent telemetry over active batches, calculates composite trust scores (0–100), and returns integrity status checkpoints.
- GET /telemetry/quarantine: Retrieves deduplicated audit trail records and IsoLog SHA-256 hash chains for forensic inspection.
- POST /demo/scenario/{id}: Orchestrates the 5 interactive demonstration scenarios (golden-path, transit-degradation, thermal-quarantine, sensor-replay-fraud, fefo-discount), updating state across all roles simultaneously.

backend/chain.py — Web3 Blockchain Connector & Struct Packer

• Key Imports: web3 (Web3, HTTPProvider), eth_account (Account), json, pathlib.
• Core Functions & Logic:
- pack_condition(temp, humidity, lat, lon, flags): Bit-packs 5 sensor metrics into a single 256-bit unsigned integer (uint256) using bitwise shifts (<<) and bitwise OR (|), enabling 1-slot SSTORE gas optimization.
- record_condition_on_chain(...): Loads the deployer private key, estimates gas, signs the raw transaction, and

invokes ColdChainMonitor.recordConditionPacked().
- transfer_custody(...): Interacts with CustodyTransfer.sol to advance batch states forward while enforcing strict role permissions.

backend/db.py — Database Engine & Schema Definitions

• Key Imports: sqlite3, pathlib, contextlib.
• Core Functions & Tables:
- init_db(): Initializes relational tables: 'readings' (raw telemetry storage), 'batches' (produce metadata), 'quarantine' (quarantined records), and 'audit_trail' (SHA-256 hash links).
- get_connection(): Thread-safe SQLite connection factory with row-factory mappings.
- reset_system_data(): Resets demonstrator state for clean faculty presentations.

3.2 AI Trust Layer Microservices (trust-layer/app/)

trust-layer/app/services/plausibility.py — Deterministic Physics Engine

• Function run_plausibility_checks(current, previous, is_transit):
- Evaluates temperature rate of change: |current.temp - previous.temp| / elapsed_min <= 1.5°C/min.
- Evaluates humidity rate of change: |current.hum - previous.hum| / elapsed_min <= 5.0%/min.
- Implements the Haversine formula to calculate great-circle distance over GPS coordinates and enforces velocity <= 120 km/h.
- Refined to ensure benign ambient fluctuations within configured crop thresholds do not generate false rejections.

trust-layer/app/services/anomaly_detector.py — Isolation Forest ML Engine

• Class AnomalyDetectorWrapper (Scikit-Learn IsolationForest):
- Initialized with hyperparameters: n_estimators=100, contamination=0.05, max_samples=256.
- Uses recursive random feature partitioning. Points isolated at shallow tree path lengths are classified as anomalies.
- Emits anomaly boolean flag and normalized continuous anomaly score in [0.0, 1.0].

trust-layer/app/services/policy.py — Dynamic Biological Crop Policy

• Function get_crop_policy(crop_type):
- Retrieves biological limits: Tomato (2.0°C–8.0°C, Spoilage > 15.0°C), Mango (10.0°C–15.0°C), Milk (1.0°C–4.0°C), Grains (15.0°C–25.0°C).
- Supports exact and substring matching with standardized fallback profiles for unknown cultivars.

trust-layer/app/services/integrity.py — Cryptographic Authenticator

• Function verify_telemetry_integrity(payload, registered_sensor_address):
- Recovers the public key from the ECDSA secp256k1 signature and verifies sensor hardware ownership.
- Rejects stale or duplicate packets to prevent replay attacks.

trust-layer/app/services/audit.py — IsoLog SHA-256 Hash Chainer

• Class AuditService:
- Implements off-chain Merkle/chain hashing: entry_hash = SHA256(prev_hash + reading_id + fault_reasons + timestamp).
- Mathematically proves that quarantined incident logs have not been altered or deleted after the fact.

3.3 Smart Contract Architecture (contracts/contracts/)

ProductRegistry.sol (RBAC & Identity)

• Solidity ^0.8.28; inherits OpenZeppelin AccessControl and Pausable.
• Functions: registerBatch(), setBatchDocumentUri(), getBatch().
• Restricts batch registration strictly to accounts holding FARMER_ROLE. Anchors IPFS document CIDs (bytes32) permanently on-chain.

CustodyTransfer.sol (Forward-Only State Machine & Escrow)

• Governs strict sequential lifecycle transitions: REGISTERED (0) -> IN_TRANSIT (1) -> IN_STORAGE (2) -> AT_RETAIL (3) -> SOLD (4).
• Functions: initializeCustody(), transferCustody(). Reverts on backward transitions or unauthorized callers.
• Records stage price additions, establishing an immutable fair price distribution trail from farmer to retail.

ColdChainMonitor.sol (Bit-Packed Telemetry & SLA Engine)

• Stores bit-packed condition words submitted by the authorized ORACLE_ROLE.
• Functions: recordConditionPacked(), recordConditionsBatchPacked(), getConditionPacked().
• Dynamic SLA Calculation: Checks continuous excursion durations against policy limits; logs penalties against carrier escrow payouts.

PolicyConfig.sol & AccessControlRoles.sol

• PolicyConfig.sol: Decoupled on-chain crop policy registry allowing authorized agronomists to update biological temperature bounds without redeploying logic contracts.
• AccessControlRoles.sol: Defines standardized role constants (DEFAULT_ADMIN_ROLE, FARMER_ROLE, LOGISTICS_ROLE, RETAILER_ROLE, ORACLE_ROLE).

Section 4: Technical Deep-Dives, Mathematical Models & Gas Optimization

4.1 The Blockchain Oracle Dilemma (GIGO)

Blockchains provide absolute immutability and decentralized consensus, but they are blind to the physical world outside their virtual machine. This architectural vulnerability is known as the Garbage-In, Garbage-Out (GIGO) Oracle Problem. If a corrupted, drifted, or malicious IoT sensor submits false temperature readings, the blockchain faithfully commits that falsehood forever. AgriChain solves this by placing an AI Trust Layer as an off-chain pre-oracle gatekeeper, ensuring only mathematically and physically verified data ever reaches the ledger.

4.2 Isolation Forest Mathematical Formulation

The Isolation Forest operates on the principle that anomalies are few and structurally divergent. In an ensemble of Isolation Trees (iTrees), anomalies require very few random splits to become isolated in terminal nodes. The anomaly score s(x, n) is defined as:

Mathematical Model: Isolation Forest Path Length
Anomaly Score Formula: $s(x, n) = 2^{(-E(h(x)) / c(n))}$

Where h(x) is the path length in a tree, E(h(x)) is the average path length across 100 trees, and c(n) is the average path length of unsuccessful searches in a Binary Search Tree:
$c(n) = 2 * \ln(n - 1) + 0.5772156649$ (Euler's Constant) $- (2 * (n - 1) / n)$

Decision Criteria: If s -> 1.0 (E(h(x)) -> 0), the reading is a definite anomaly. If s < 0.5, the reading is normal.

4.3 Single-Slot Bitwise Packed Struct (58% Gas Reduction)

In Ethereum, modifying an uninitialized storage slot (SSTORE) costs 20,000 gas. Storing 5 individual variables (timestamp, temperature, humidity, latitude, longitude) across separate 256-bit slots costs over 105,000 gas per reading, making cold-chain tracking economically non-viable. AgriChain packs all 5 parameters into a single 256-bit word:

Bit Range	Field Name	Solidity Type	Precision / Representation

Bits [0 - 63] (64 bits)	timestamp	uint64	Unix epoch timestamp in seconds.
Bits [64 - 79] (16 bits)	temp_deci_c	int16	Temperature in deci-degrees Celsius (e.g. 4.5°C = 45).
Bits [80 - 95] (16 bits)	humidity_deci_pct	uint16	Humidity in deci-percent (e.g. 88.5% = 885).
Bits [96 - 127] (32 bits)	latitude	uint32	Micro-degrees (degrees * 10^6).
Bits [128 - 159] (32 bits)	longitude	uint32	Micro-degrees (degrees * 10^6).
Bits [160 - 167] (8 bits)	flags	uint8	Bitmask: Bit 0 = Breach, Bit 1 = Replay, Bit 2 = Quarantined.
Bits [168 - 255] (88 bits)	reserved	uint88	Allocated for future protocol expansion.

Impact: Batched writes execute at ~43,570 gas per reading, delivering a 58.5% reduction in blockchain transaction costs.

4.4 Strict Zero Floating-Point Rule

Neither the Ethereum Virtual Machine (EVM) nor Solidity supports IEEE 754 floating-point operations. Using floats in Python or JavaScript creates rounding discrepancies and potential financial exploit vectors. AgriChain strictly enforces fixed-point integer scaling:

• Currency: Stored strictly as integer paise (1 INR = 100 paise). A price of INR 25.50 is stored as 2550.

• Temperature: Stored strictly as integer deci-degrees Celsius (1°C = 10 deci-°C). A temperature of 4.5°C is stored as 45.

Section 5: Comprehensive Viva Voce Examination Q&A

Below are 8 core questions frequently asked by external university examiners, accompanied by structured, high-scoring technical answers.

Q1: Why do we need an AI Trust Layer if the blockchain is already immutable?

Answer: Blockchain guarantees data immutability, but NOT data truthfulness. This is the classic Oracle Problem (Garbage-In, Garbage-Out). If a faulty or tampered IoT sensor transmits a falsified reading of 4°C when food is spoiling at 25°C, the blockchain will faithfully record that lie permanently. Our AI Trust Layer serves as a pre-oracle gatekeeper, validating physical plausibility and ML anomaly scores to quarantine bad data before it ever reaches the ledger.

Q2: Why did you choose Isolation Forest instead of Deep Learning (LSTM, Autoencoder, or CNN)?

Answer: Three critical engineering justifications: (1) Unsupervised Nature: Anomalies in cold-chain logistics are rare, diverse, and lack massive labeled training datasets. Isolation Forest requires no labeled anomaly ground truth. (2) Real-Time Latency: Deep learning inference introduces significant latency and requires GPU acceleration. Isolation Forest executes in under 15ms on standard CPU hardware. (3) Linear Complexity O(n): It scales linearly with sample size, making it suitable for high-frequency IoT streaming gateways.

Q3: Why not run the AI model directly inside the Solidity smart contract on-chain?

Answer: Running machine learning on-chain is economically and technically impossible: (1) Gas Costs: Iterating through 100 decision trees in EVM opcodes would cost thousands of dollars per reading. (2) Lack of Floating-Point

Math: The EVM lacks hardware floating-point instructions required for logarithmic path calculations. (3) Latency: Blockchain transaction block finality takes seconds or minutes, while IoT ingestion requires sub-second responses.

Q4: How does the system defend against sensor spoofing or replay attacks?

Answer: Through a three-layer defense: (1) Strictly Monotonic Timestamps: Every incoming reading must have a timestamp strictly greater than the preceding reading (t_k > t_{k-1}); stale packets are dropped. (2) ECDSA Hardware Signatures: Readings are signed using the sensor's secure hardware key; the backend recovers the signer address via eth-account. (3) Haversine Kinematic Checks: Replaying historical coordinates from another region triggers our velocity check (> 120 km/h), quarantining the packet for impossible teleportation.

Q5: What happens if a critical vulnerability is discovered in your deployed smart contracts?

Answer: We integrated the OpenZeppelin Pausable Circuit Breaker pattern across our contracts (ProductRegistry, CustodyTransfer, ColdChainMonitor). An authorized administrator can execute an emergency pause, instantly freezing custody handoffs and escrow releases while preserving state integrity until an audit or contract upgrade is completed.

Q6: Why is farmer payment calculated in paise rather than Ether or ERC-20 tokens?

Answer: Agricultural producers operating in rural regions cannot absorb cryptocurrency volatility. A drop in ETH price between harvest and delivery would bankrupt a farmer. By calculating escrow settlements in integer paise (1 INR = 100 paise), we guarantee deterministic, fiat-equivalent compensation with zero floating-point rounding errors and zero currency risk.

Q7: How do you handle cold-chain SLA penalties automatically?

Answer: ColdChainMonitor.sol continuously evaluates verified temperatures against the biological limits configured in PolicyConfig.sol. If temperature exceeds allowable thresholds for more than 15 consecutive minutes, the contract automatically records a thermal breach and deducts a dynamic financial penalty from the carrier's escrow payout upon delivery.

Q8: How does a consumer verify the authenticity of a produce batch?

Answer: Consumers scan a QR code printed on the packaging using their mobile device. The QR code deep-links to our public Consumer Provenance Portal, querying the blockchain to render the complete 5-stage geographical timeline, temperature history, dynamic freshness grade (A+), fair price distribution per kg, and cryptographic digital certificate anchored to IPFS.

Section 6: Quick Code Location Cheat Sheet ('Where is it in the code?')

Use this cheat sheet during viva demonstrations when the examiner asks to inspect specific implementation code:

Technical Concept	File Path in Codebase	Function / Class Name	Key Logic Inspected
Isolation Forest ML	trust-layer/app/services/anomaly_detector.py	AnomalyDetectorWrapper	Scikit-Learn IsolationForest(n_estimators=100, contamination=0.05).
Thermal Physics Rules	trust-layer/app/services/plausibility.py	run_plausibility_checks()	Checks |ΔT/Δt| <= 1.5°C/min, humidity rate, and Haversine velocity.
Crop Biological	trust-layer/app/services/policy.py	get_crop_policy()	Biological temperature thresholds for tomatoes,

Policies			mangoes, milk, etc.
ECDSA Hardware Signature	trust-layer/app/services/integrity.py	verify_telemetry_integrity()	Recovers public address via eth-account and validates monotonic timestamps.
SHA-256 IsoLog Hash Chain	trust-layer/app/services/audit.py	AuditService.record_quarantine()	entry_hash = SHA256(prev_hash + reading_id + fault_reasons + timestamp).
Bitwise Struct Packing	backend/chain.py	pack_condition()	Packs 5 variables into 256 bits via bitwise shifts (<<) and bitwise OR (|).
Web3 Blockchain Dispatch	backend/chain.py	record_condition_on_chain()	Dispatches signed transactions to ColdChainMonitor.sol on EVM.
Forward Custody FSM	contracts/contracts/CustodyTransfer.sol	transferCustody()	Enforces REGISTERED -> IN_TRANSIT -> IN_STORAGE -> AT_RETAIL -> SOLD.
Pausable Circuit Breakers	contracts/contracts/ProductRegistry.sol	pause(), unpause()	OpenZeppelin Pausable emergency circuit breaker toggles.
Raw Telemetry Database	backend/db.py	init_db(), get_connection()	SQLite schema: readings, batches, quarantine, and audit_trail.
Live AI Sandbox UI	frontend/src/Farmer/Views/AITrustView.jsx	AITrustView component	Interactive sliders, live Trust Score card, real-time quarantine audit stream.
Demonstration Scenarios	frontend/src/components/ShowcaseDrawer.jsx	ShowcaseDrawer component	1-click faculty demo lab executing all 5 real-world scenarios.